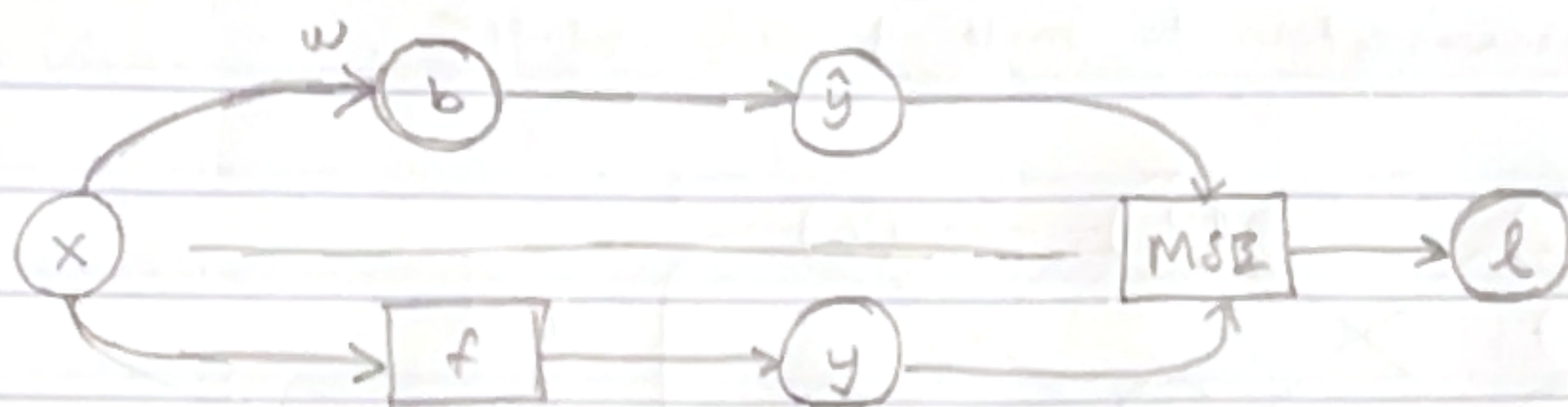


# Neural Network from scratch



$$y = F(x)$$

Goal: min  $l$

$$\hat{y} = wx + b$$

Solution: update  $w, b$  in direction of decreasing  $l$  ( $\Delta X \propto -\frac{\partial l}{\partial X}$ )

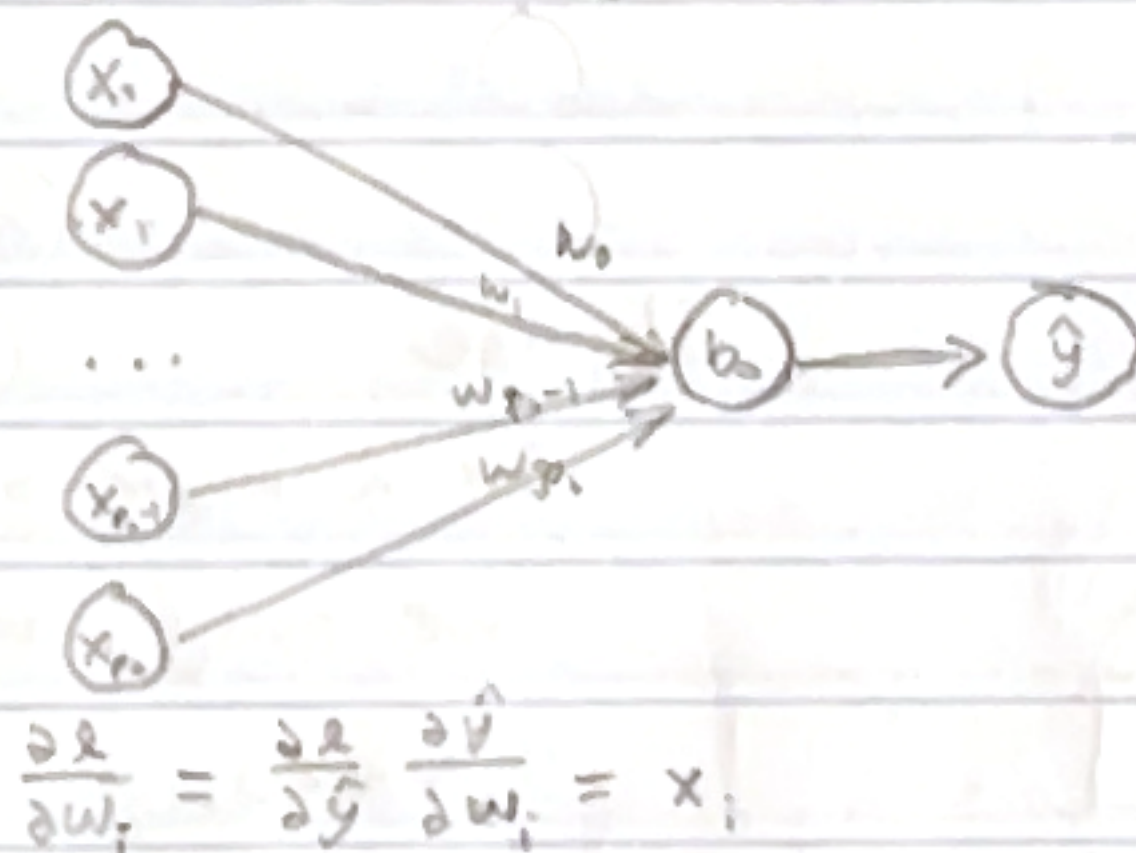
$$l = (y - \hat{y})^2$$

So how do we find  $\frac{\partial l}{\partial w}$  &  $\frac{\partial l}{\partial b}$ ?

Chain rule.

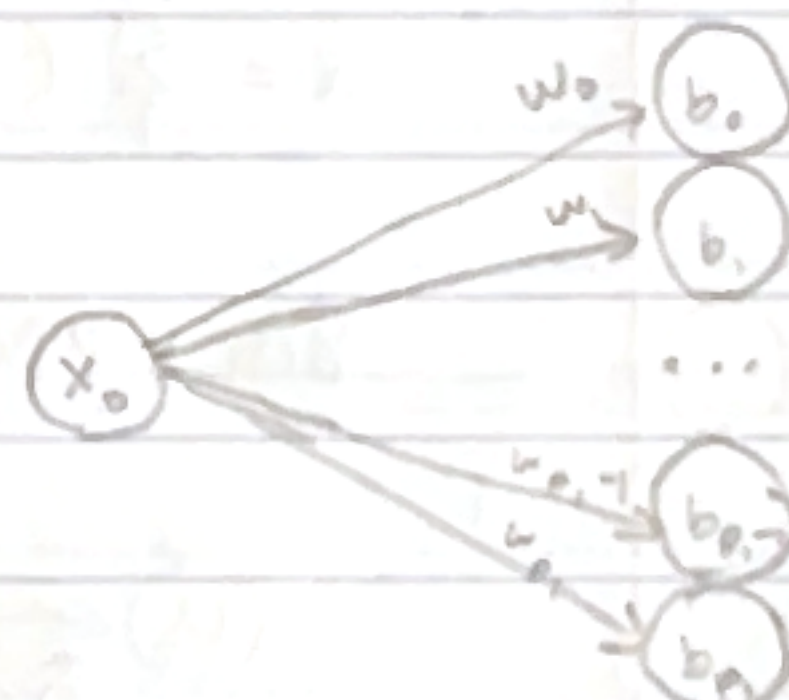
$$\frac{\partial l}{\partial w} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w} \quad \frac{\partial l}{\partial b} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b}$$

Extrapolate to  $P_0$  inputs



$$\frac{\partial l}{\partial w_i} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_i} = x_i$$

Extrapolate to  $P_1$  outputs

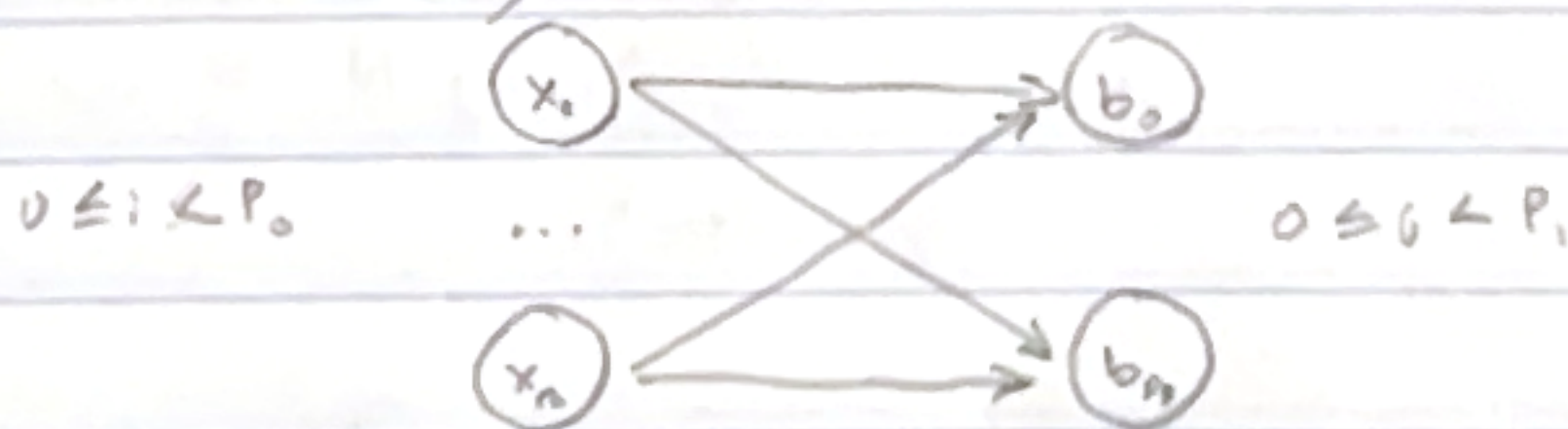


$$\frac{\partial l}{\partial w_i} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_i} = x_0$$

Note that when nodes share the same input their partial derivatives are all the input.

Now,  $P_0$  inputs and  $P_1$  outputs

Notation:



$w_{ij}$  - weight of  $i$ th input node and  $j$ th output node

$$\frac{\partial l}{\partial w_{0j}} = \frac{\partial l}{\partial w_{01}} = \frac{\partial l}{\partial w_{0j}} = x_0$$

we need to find  $\frac{\partial l}{\partial w_{ij}}$

$$\frac{\partial l}{\partial w_{ij}} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_{ij}} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_i} = x_i$$

attached to the same input

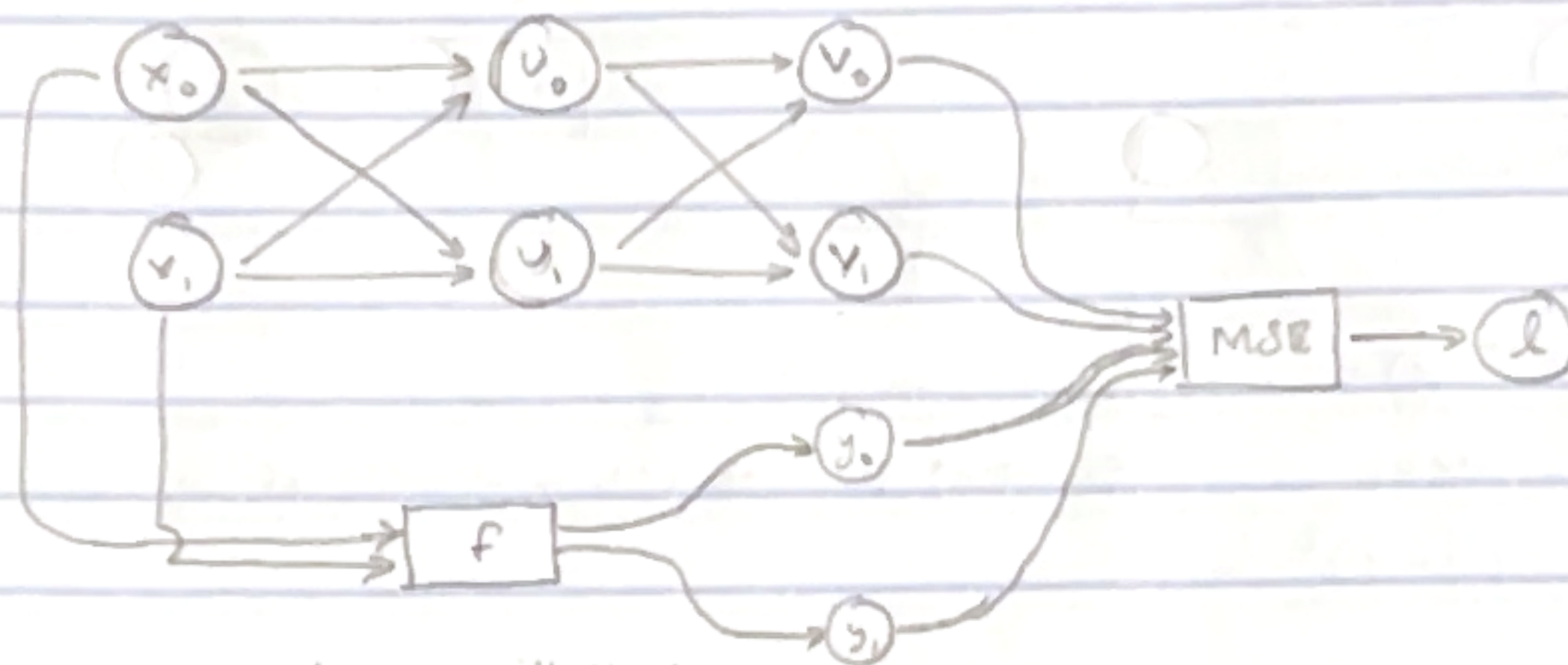
Also,  $\frac{\partial l}{\partial b_j}$

$$\frac{\partial l}{\partial b_j} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b_j} = 1$$

not affected by  $x$



Further generalization to multi-layered network...



with element of  $U$   
with element of  $U$  & bias  $b$

$$u_i = U \cdot w_{ij} x_j + U \cdot b_j$$

vector notation

$$U = U \cdot W X + U \cdot b$$

$$V = V \cdot W U + V \cdot b$$

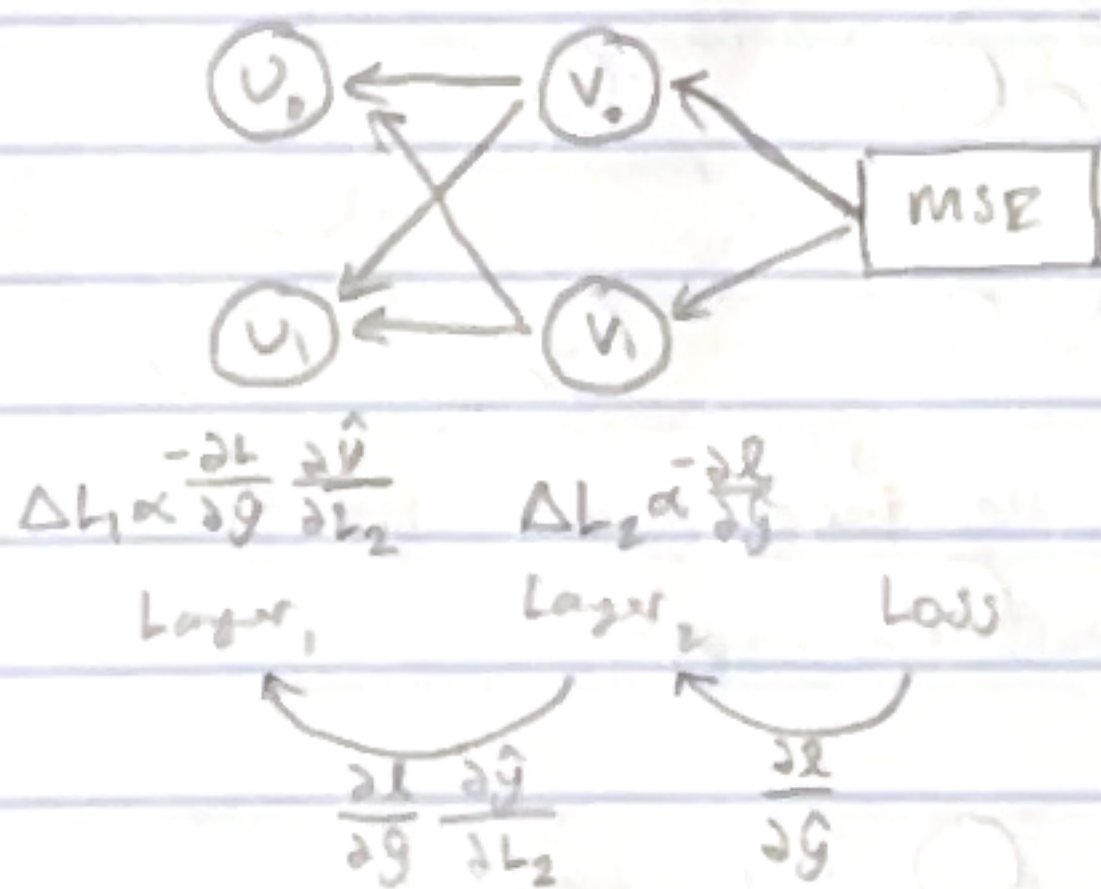
$$l = \sum_i (y_i - v_i)^2$$

When we run a call through the network in training mode, we must store the partial derivatives at each section.

## BACKPROP

## Storage

For a layer of  $p_0$  inputs and  $p_1$  outputs we only need  $p_0$  to store weights and biases.



$$[x] \rightarrow [y]$$

$$[1, p_0] \quad [1, p_1]$$

For batches of  $N$ , we sum the gradients and divide by  $N$  to get an "average" gradient.

$$\Delta W \propto - \text{ones}[p_0, p_1] \cdot \text{reduce\_sum}(x, 0, T) \cdot \Delta \text{prev}$$

$$[p_0, p_1] \quad [p_0, p_1] \quad [p_0, 1] \quad [N, p_0] \quad [1, p_1]$$

$$\Delta b \propto - \text{ones}[1, p_1] \cdot \Delta \text{prev}$$

$$[1, p_1] \quad [1, p_1] \quad [1, p_1]$$